

Manual Completo de Programación

UIFlow 2.0 + MicroPython

M5StickC Plus 2

APS-GRIA

Universidad de Vigo — Curso 2025-2026

Referencia completa de todas las funciones disponibles



1. Introducción

UIFlow 2.0 es el entorno de desarrollo oficial de M5Stack basado en MicroPython. Este manual documenta TODAS las funciones disponibles en el M5StickC Plus 2, obtenidas directamente del firmware mediante `dir()`.

1.1 Estructura básica de un programa

```
import os, sys, io
import M5
from M5 import *

def setup():
    M5.begin()

def loop():
    M5.update()

if __name__ == "__main__":
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print("Error:", e)
```

✓ **TIP:** `M5.begin()` inicializa todo el hardware. `M5.update()` actualiza botones y pantalla. Ambos son obligatorios.

1.2 Imports principales

<code>import M5</code>	# Sistema base
<code>from M5 import *</code>	# Widgets, Botones, Imu, Speaker, Power...
<code>from hardware import *</code>	# Pin, I2C, ADC, PWM, UART (import individual)
<code>import time</code>	# Temporización
<code>import network</code>	# WiFi
<code>import socket</code>	# Comunicación red
<code>from umqtt import MQTTClient</code>	# MQTT
<code>import ssl</code>	# Conexiones seguras SSL/TLS
<code>import json</code>	# Serialización JSON
<code>import math</code>	# Funciones matemáticas
<code>import struct</code>	# Empaquetado binario
<code>import gc</code>	# Garbage collector
<code>import os</code>	# Sistema de archivos

2. M5 — Sistema Base

Módulo principal que da acceso a todo el hardware del dispositivo.

Función / Atributo	Retorno	Descripción
M5.begin()	void	Inicializa todo el hardware (OBLIGATORIO al inicio)
M5.update()	void	Actualiza botones, pantalla e inputs (OBLIGATORIO en loop)
M5.end()	void	Finaliza y libera hardware
M5.getBoard()	int	Identificador del tipo de placa
M5.getDisplay()	Display	Obtiene el display principal
M5.getDisplayCount()	int	Número de displays conectados
M5.getDisplayIndex()	int	Índice del display actual
M5.addDisplay(display)	void	Añade un display externo
M5.setPrimaryDisplay(idx)	void	Define el display principal
M5.createSpeaker(config)	Speaker	Crea instancia de speaker con config
M5.createMic(config)	Mic	Crea instancia de micrófono con config

Submódulos accesibles desde M5

Función / Atributo	Retorno	Descripción
M5.BtnA / BtnB / BtnPWR	Button	Botones (ver sección 4)
M5.Display / M5.Lcd	Display	Pantalla (acceso directo)
M5.Speaker	Speaker	Altavoz (ver sección 5)
M5.Imu	IMU	Acelerómetro y giroscopio (ver sección 6)
M5.Power	Power	Gestión de energía (ver sección 7)
M5.Mic	Mic	Micrófono
M5.Led	Led	LED interno
M5.Als	ALS	Sensor de luz ambiental
M5.Widgets	Widgets	Sistema de widgets (ver sección 3)
M5.Touch	Touch	Pantalla táctil (no disponible en StickC Plus 2)
M5.BOARD	int	Constante identificadora de la placa

```
# Ejemplo: Setup mínimo
import M5
from M5 import *

def setup():
    M5.begin()
    print("Placa:", M5.getBoard())
    print("Displays:", M5.getDisplayCount())
```

3. Widgets — Pantalla y Gráficos

El M5StickC Plus 2 tiene pantalla TFT de 135×240 píxeles. Colores en formato 0xRRGGBB (24 bits).

3.1 Funciones globales de Widgets

Función / Atributo	Retorno	Descripción
Widgets.fillScreen(color)	void	Rellena toda la pantalla con un color
Widgets.setBrightness(nivel)	void	Brillo de pantalla (0-255)
Widgets.setRotation(rot)	void	Rotación: 0, 1, 2, 3 (x90°)

```
Widgets.fillScreen(0x222222) # Fondo gris oscuro
Widgets.setBrightness(128)   # Brillo al 50%
Widgets.setRotation(1)       # Rotar 90° (landscape)
```

3.2 Widgets.FONTS — Fuentes disponibles

Función / Atributo	Retorno	Descripción
Widgets.FONTS.DejaVu12	Font	12px — ~22 caracteres/línea
Widgets.FONTS.DejaVu18	Font	18px — ~15 caracteres/línea
Widgets.FONTS.DejaVu24	Font	24px — ~11 caracteres/línea

3.3 Widgets.COLOR — Colores predefinidos

```
# Colores más usados (hex RGB 24-bit)
0xFF0000 # Rojo      0x00FF00 # Verde      0x0000FF # Azul
0xFFFF00 # Amarillo  0xFF8800 # Naranja   0x00FFFF # Cian
0xFF00FF # Magenta   0xFFFFFF # Blanco    0x000000 # Negro
0x222222 # Gris oscuro (fondo recomendado)
```

3.4 Widgets.Title(texto, x, color_texto, color_fondo, fuente)

Crea una barra de título en la parte superior de la pantalla.

Función / Atributo	Retorno	Descripción
texto	str	Texto del título
x	int	Posición X (normalmente 3)
color_texto	int	Color del texto (hex)
color_fondo	int	Color de fondo de la barra (hex)
fuelle	Font	Widgets.FONTS.DejaVuXX

```
title = Widgets.Title("Mi App", 3, 0xFFFFFF, 0x0000FF, Widgets.FONTS.DejaVu18)
```

3.5 Widgets.Label(texto, x, y, escala, color, fondo, fuente)

Etiqueta de texto posicionable. Es el widget más usado para mostrar información.

Función / Atributo	Retorno	Descripción
.setText(texto)	void	Cambia el texto mostrado
.setColor(color, fondo)	void	Cambia colores de texto y fondo

<code>.setVisible(bool)</code>	void	Muestra u oculta la etiqueta
<code>.setFont(fuente)</code>	void	Cambia la fuente

```
# Crear label
lbl = Widgets.Label("Hola", 5, 50, 1.0, 0x00FF00, 0x222222, Widgets.FONTS.DejaVu18)

# Actualizar
lbl.setText("Temp: 23.5 C")
lbl.setColor(0xFF0000, 0x000000) # Rojo sobre negro
lbl.setVisible(False)           # Ocultar
lbl.setVisible(True)            # Mostrar
```

3.6 Widgets.Rectangle(x, y, ancho, alto, color, color_relleno)

Dibuja un rectángulo en pantalla.

```
# Rectángulo relleno verde
rect = Widgets.Rectangle(10, 40, 100, 50, 0x00FF00, 0x00FF00)

# Rectángulo solo borde (contorno rojo, sin relleno)
rect2 = Widgets.Rectangle(10, 100, 100, 50, 0xFF0000, 0x222222)
```

3.7 Widgets.Circle(x_centro, y_centro, radio, color, color_relleno)

Dibuja un círculo.

```
# Círculo azul relleno
circ = Widgets.Circle(67, 120, 30, 0x0000FF, 0x0000FF)

# Actualizar posición o color si es necesario
# circ.setColor(0xFF0000, 0xFF0000)
```

3.8 Widgets.Line(x1, y1, x2, y2, color)

Dibuja una línea entre dos puntos.

```
# Línea diagonal amarilla
line = Widgets.Line(0, 0, 135, 240, 0xFFFF00)

# Línea horizontal
line2 = Widgets.Line(0, 120, 135, 120, 0xFFFF00)
```

3.9 Widgets.Triangle(x1, y1, x2, y2, x3, y3, color, color_relleno)

Dibuja un triángulo definido por 3 vértices.

```
# Triángulo verde
tri = Widgets.Triangle(67, 40, 30, 120, 104, 120, 0x00FF00, 0x00FF00)
```

3.10 Widgets.Image(ruta, x, y)

Muestra una imagen desde la flash.

```
# Mostrar imagen PNG o BMP
img = Widgets.Image("/flash/logo.png", 10, 30)

# Cambiar imagen
# img.setImage("/flash/otro.png")
```

3.11 Widgets.QRCode(texto, x, y, tamaño)

Genera y muestra un código QR en pantalla.

```
# QR con URL
qr = Widgets.QRCode("https://uvigo.es", 17, 40, 100)
```

```
# QR con texto
qr2 = Widgets.QRCode("Hola Mundo", 17, 40, 100)
```

✓ **TIP:** El QR se adapta al tamaño especificado. Para URLs cortas, un tamaño de 100px es suficiente.

Ejemplo completo: Dashboard con widgets gráficos

```
def setup():
    M5.begin()
    Widgets.fillScreen(0x222222)
    Widgets.setBrightness(200)

    # Título
    Widgets.Title("Dashboard", 3, 0xFFFFFF, 0x1A5276, Widgets.FONTS.DejaVu18)

    # Rectángulo de fondo para datos
    Widgets.Rectangle(5, 30, 125, 60, 0x444444, 0x333333)

    # Labels de datos
    global lbl_temp, lbl_hum
    lbl_temp = Widgets.Label("T: --", 10, 40, 1.0, 0x00FF00, 0x333333, Widgets.FONTS.DejaVu18)
    lbl_hum = Widgets.Label("H: --", 10, 65, 1.0, 0x00FFFF, 0x333333, Widgets.FONTS.DejaVu18)

    # Línea separadora
    Widgets.Line(5, 95, 130, 95, 0x555555)

    # Indicador circular (estado)
    global circ_status
    circ_status = Widgets.Circle(67, 120, 15, 0xFF0000, 0xFF0000)
```

4. Botones

El M5StickC Plus 2 tiene 3 botones: BtnA (frontal), BtnB (lateral) y BtnPWR (encendido). BtnC y BtnEXT existen en el API pero no están disponibles físicamente.

⚠ Llamado a `M5.update()` en cada iteración del loop es OBLIGATORIO para que los botones funcionen.

4.1 Métodos de detección de pulsación

Función / Atributo	Retorno	Descripción
<code>.wasPressed()</code>	bool	True si se pulsó desde el último update
<code>.wasReleased()</code>	bool	True si se soltó desde el último update
<code>.isPressed()</code>	bool	True si está pulsado ahora mismo
<code>.isReleased()</code>	bool	True si está suelto ahora mismo
<code>.isHolding()</code>	bool	True si se mantiene pulsado (pulsación larga activa)
<code>.wasHold()</code>	bool	True si se completó una pulsación larga
<code>.wasClicked()</code>	bool	True si hubo un click (pulsar y soltar)
<code>.wasSingleClicked()</code>	bool	True si fue un click simple (no doble)
<code>.wasDoubleClicked()</code>	bool	True si fue un doble click
<code>.wasChangePressed()</code>	bool	True si cambió a estado pulsado

4.2 Métodos de información y configuración

Función / Atributo	Retorno	Descripción
<code>.getClickCount()</code>	int	Número de clicks detectados
<code>.wasDecidedClickCount()</code>	bool	True si ya se resolvió el conteo de clicks
<code>.pressedFor()</code>	int	Milisegundos que lleva pulsado
<code>.releasedFor()</code>	int	Milisegundos desde que se soltó
<code>.lastChange()</code>	int	Timestamp del último cambio de estado (ms)
<code>.setDebounceThresh(ms)</code>	void	Define umbral anti-rebote en ms
<code>.setHoldThresh(ms)</code>	void	Define umbral para considerar hold (ms)

4.3 Callbacks

Función / Atributo	Retorno	Descripción
<code>.setCallback(tipo, funcion)</code>	void	Registra función callback para un evento
<code>.removeCallback(tipo)</code>	void	Elimina callback de un tipo
<code>.CB_TYPE</code>	enum	Tipos de callback disponibles

```
# Método 1: Polling (comprobar en el loop)
def loop():
    M5.update()
    if BtnA.wasPressed():
        print("A pulsado!")
    if BtnA.wasHold():
        print("A mantenido!")
    if BtnA.wasDoubleClicked():
        print("A doble click!")
```

```
# Método 2: Callbacks
def on_btnA_press():
```

```

    print("Callback: A pulsado!")

# Registrar callback (depende de la versión de firmware)
# BtnA.setCallback(BtnA.CB_TYPE.WAS_PRESSED, on_btnA_press)

```

Ejemplo: Menú interactivo con clicks

```

opciones = ["Medir", "Enviar", "Config", "Salir"]
idx = 0
lbl_menu = None
lbl_info = None

def setup():
    global lbl_menu, lbl_info
    M5.begin()
    Widgets.fillScreen(0x222222)
    Widgets.Title("Menu", 3, 0xFFFFFF, 0x0000FF, Widgets.FONTS.DejaVu18)
    lbl_menu = Widgets.Label("> " + opciones[0], 5, 35, 1.0,
                             0x00FF00, 0x222222, Widgets.FONTS.DejaVu18)
    lbl_info = Widgets.Label("A=nav B=ok", 5, 65, 1.0,
                             0x888888, 0x222222, Widgets.FONTS.DejaVu12)

def loop():
    global idx
    M5.update()

    if BtnA.wasPressed():          # Navegar
        idx = (idx + 1) % len(opciones)
        lbl_menu.setText("> " + opciones[idx])

    if BtnA.wasDoubleClicked():    # Navegar atrás
        idx = (idx - 1) % len(opciones)
        lbl_menu.setText("> " + opciones[idx])

    if BtnB.wasPressed():          # Seleccionar
        lbl_info.setText("OK: " + opciones[idx])
        M5.Speaker.tone(1500, 80)

    if BtnPWR.wasPressed():        # Acción especial
        lbl_info.setText("PWR pulsado!")

    time.sleep_ms(20)

```


5. Speaker (Altavoz)

Altavoz integrado accesible a través de M5.Speaker.

5.1 Métodos completos del Speaker

Función / Atributo	Retorno	Descripción
.tone(freq, duracion)	void	Emite tono. freq en Hz, duración en ms
.stop()	void	Detiene cualquier sonido en reproducción
.begin()	void	Inicializa el speaker (normalmente automático)
.end()	void	Finaliza el speaker y libera recursos
.config(cfg)	void	Aplica configuración personalizada
.setVolume(vol)	void	Volumen absoluto (0-255)
.getVolume()	int	Obtiene volumen actual (0-255)
.setVolumePercentage(pct)	void	Volumen en porcentaje (0-100)
.getVolumePercentage()	int	Obtiene volumen en porcentaje
.setChannelVolume(ch, vol)	void	Volumen de un canal específico
.getChannelVolume(ch)	int	Obtiene volumen de un canal
.setAllChannelVolume(vol)	void	Volumen de todos los canales
.getPlayingChannels()	int	Número de canales reproduciendo
.isPlaying()	bool	True si hay sonido reproduciéndose
.isRunning()	bool	True si el speaker está activo
.isEnabled()	bool	True si el speaker está habilitado
.playRaw(data, rate, stereo, repeat)	void	Reproduce audio raw (PCM)
.playWav(data, repeat)	void	Reproduce datos WAV desde buffer
.playWavFile(path, ch, repeat)	void	Reproduce archivo WAV desde flash
.setPA(enable)	void	Habilita/deshabilita amplificador de potencia

```
# Beeps básicos
M5.Speaker.tone(1000, 200)    # 1kHz, 200ms
M5.Speaker.tone(3000, 80)     # Agudo corto (notificación)
M5.Speaker.tone(500, 500)     # Grave largo (error)

# Control de volumen
M5.Speaker.setVolumePercentage(50) # 50%
print("Vol:", M5.Speaker.getVolumePercentage(), "%")

# Melodía: escala Do-Re-Mi
notas = [262, 294, 330, 349, 392, 440, 494, 523]
for nota in notas:
    M5.Speaker.tone(nota, 200)
    time.sleep_ms(250)

# Reproducir archivo WAV
# M5.Speaker.playWavFile("/flash/sonido.wav")

# Verificar estado
if M5.Speaker.isPlaying():
    print("Reproduciendo...")
    M5.Speaker.stop()
```

6. IMU (Acelerómetro y Giroscopio)

MPU6886 integrado con acelerómetro 3 ejes y giroscopio 3 ejes. No requiere inicialización.

6.1 Métodos del IMU

Función / Atributo	Retorno	Descripción
<code>Imu.getAccel()</code>	tuple	Devuelve (ax, ay, az) en g (1g = 9.81 m/s ²)
<code>Imu.getGyro()</code>	tuple	Devuelve (gx, gy, gz) en grados/segundo
<code>Imu.getMag()</code>	tuple	Devuelve (mx, my, mz) magnetómetro (si disponible)
<code>Imu.getType()</code>	int	Tipo de IMU instalada
<code>Imu.isEnabled()</code>	bool	True si la IMU está activa
<code>Imu.IMU_TYPE</code>	enum	Constantes de tipos de IMU

```
# Lectura básica
acc = Imu.getAccel()    # (ax, ay, az) en g
gyro = Imu.getGyro()    # (gx, gy, gz) en dps

print("Acc: X={:.2f} Y={:.2f} Z={:.2f} g".format(acc[0], acc[1], acc[2]))
print("Gyro: X={:.1f} Y={:.1f} Z={:.1f} dps".format(gyro[0], gyro[1], gyro[2]))

# Verificar tipo
print("IMU tipo:", Imu.getType())
print("IMU activa:", Imu.isEnabled())
```

Ejemplo: Inclinómetro (pitch y roll)

```
import math

def loop():
    M5.update()
    acc = Imu.getAccel()

    pitch = math.atan2(acc[0], math.sqrt(acc[1]**2 + acc[2]**2)) * 180 / math.pi
    roll = math.atan2(acc[1], math.sqrt(acc[0]**2 + acc[2]**2)) * 180 / math.pi

    lbl_pitch.setText("Pitch: {:.1f}".format(pitch))
    lbl_roll.setText("Roll: {:.1f}".format(roll))
    time.sleep_ms(50)
```

Ejemplo: Detección de agitación (shake) y caída libre

```
import math

SHAKE_TH = 2.0    # Umbral agitación (g)
FREEFALL_TH = 0.3 # Umbral caída libre (g)

def loop():
    M5.update()
    acc = Imu.getAccel()
    mag = math.sqrt(acc[0]**2 + acc[1]**2 + acc[2]**2)

    if mag > SHAKE_TH:
        lbl.setText("SHAKE! {:.1f}g".format(mag))
        M5.Speaker.tone(2000, 100)
    elif mag < FREEFALL_TH:
        lbl.setText("CAÍDA LIBRE!")
        M5.Speaker.tone(500, 300)
    else:
        lbl.setText("Normal {:.2f}g".format(mag))
    time.sleep_ms(50)
```

Ejemplo: Registro continuo de IMU a velocidad máxima

```
# Captura rápida sin sleep para máximo sample rate
import time, struct, socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
target = ("192.168.1.100", 5005)

while True:
    ts = time.time()
    acc = Imu.getAccel()
    gyro = Imu.getGyro()
    data = struct.pack('<I6f', ts,
        acc[0], acc[1], acc[2],
        gyro[0], gyro[1], gyro[2])
    sock.sendto(data, target)
# Sin sleep: >200 muestras/segundo
```

7. Power (Energía y Batería)

Control completo de energía, batería, modos de sueño y salidas de alimentación.

7.1 Batería y carga

Función / Atributo	Retorno	Descripción
Power.getBatteryLevel()	int	Nivel de batería en % (0-100)
Power.getBatteryVoltage()	int	Voltaje de batería en mV
Power.getBatteryCurrent()	int	Corriente de batería en mA
Power.isCharging()	bool	True si está cargando por USB
Power.setBatteryCharge(enable)	void	Habilita/deshabilita carga
Power.setChargeCurrent(mA)	void	Define corriente de carga
Power.setChargeVoltage(mV)	void	Define voltaje límite de carga

7.2 USB y alimentación externa

Función / Atributo	Retorno	Descripción
Power.getVBUSVoltage()	int	Voltaje del bus USB en mV
Power.getExtVoltage()	int	Voltaje de entrada externa en mV
Power.getExtCurrent()	int	Corriente de entrada externa en mA
Power.setExtOutput(enable)	void	Habilita salida de 5V por conector externo
Power.getExtOutput()	bool	Estado de la salida externa
Power.setUsbOutput(enable)	void	Habilita salida por USB
Power.getUsbOutput()	bool	Estado de la salida USB
Power.getPortVbus()	int	Voltaje VBUS del port
Power.getPortCurrent()	int	Corriente del port
Power.setExtPortBusConfig(cfg)	void	Configura bus del port externo

7.3 Modos de sueño y apagado

Función / Atributo	Retorno	Descripción
Power.deepSleep(us)	void	Sueño profundo. us=microsegundos (0=infinito)
Power.lightSleep(us)	void	Sueño ligero. Despertar más rápido
Power.timerSleep(seconds)	void	Duerme N segundos y despierta
Power.powerOff()	void	Apaga completamente el dispositivo
Power.getKeyState()	int	Estado del botón de encendido
Power.getType()	int	Tipo de chip de gestión de energía

7.4 LED y vibración

Función / Atributo	Retorno	Descripción
Power.setLed(enable)	void	LED rojo: True=encendido, False=apagado
Power.setVibration(enable)	void	Motor de vibración (si disponible)

```
# Monitor completo de batería
bat = Power.getBatteryLevel()
volt = Power.getBatteryVoltage()
curr = Power.getBatteryCurrent()
usb_v = Power.getVBUSVoltage()
charging = Power.isCharging()

print("Batería: {}% {}mV {}mA".format(bat, volt, curr))
print("USB: {}mV Cargando: {}".format(usb_v, charging))

# Parpadeo LED como indicador
for i in range(5):
    Power.setLed(True)
    time.sleep_ms(200)
    Power.setLed(False)
    time.sleep_ms(200)

# Deep sleep 10 segundos
# Power.deepSleep(10_000_000) # 10M microsegundos = 10s

# Apagar
# Power.powerOff()
```

⚠ `deepSleep()` y `powerOff()` son irreversibles por software. El dispositivo necesita botón físico o temporizador para despertar.

8. GPIO — Pin Digital

Control de pines digitales. Importar con: `from hardware import Pin`

8.1 Constructor y métodos

Función / Atributo	Retorno	Descripción
<code>Pin(num, mode, pull)</code>	<code>Pin</code>	Crea pin. mode y pull opcionales
<code>.value()</code>	<code>int</code>	Lee estado: 0 o 1
<code>.value(v)</code>	<code>void</code>	Escribe estado: 0 o 1
<code>.on()</code>	<code>void</code>	Pone HIGH (1)
<code>.off()</code>	<code>void</code>	Pone LOW (0)
<code>.toggle()</code>	<code>void</code>	Invierte el estado actual
<code>.init(mode, pull)</code>	<code>void</code>	Reinicializa el pin
<code>.irq(handler, trigger)</code>	<code>void</code>	Configura interrupción

8.2 Constantes de Pin

Constante	Descripción
<code>Pin.IN</code>	Modo entrada
<code>Pin.OUT</code>	Modo salida
<code>Pin.OPEN_DRAIN</code>	Modo open drain (salida)
<code>Pin.PULL_UP</code>	Resistencia pull-up interna
<code>Pin.PULL_DOWN</code>	Resistencia pull-down interna
<code>Pin.IRQ_RISING</code>	Interrupción en flanco de subida
<code>Pin.IRQ_FALLING</code>	Interrupción en flanco de bajada
<code>Pin.WAKE_HIGH</code>	Despertar de sleep con nivel alto
<code>Pin.WAKE_LOW</code>	Despertar de sleep con nivel bajo
<code>Pin.DRIVE_0</code> a <code>DRIVE_3</code>	Niveles de corriente de salida

```
from hardware import Pin

# Salida digital (LED externo en GPIO26)
led = Pin(26, Pin.OUT)
led.on()           # Encender
led.off()          # Apagar
led.toggle()       # Invertir
led.value(1)       # Encender (alternativa)
print(led.value()) # Leer estado actual

# Entrada con pull-up
boton = Pin(36, Pin.IN, Pin.PULL_UP)
if boton.value() == 0:
    print("Pulsado!")

# Interrupción
def on_change(pin):
    print("Pin cambió!")

sensor = Pin(36, Pin.IN)
sensor.irq(trigger=Pin.IRQ_FALLING, handler=on_change)

# Pines disponibles M5StickC Plus 2:
# GPIO32 (Port A SDA), GPIO33 (Port A SCL)
```

GPIO26 (Hat), GPIO36 (Hat, solo entrada), GPIO00 (Hat)

⚠ GPIO36 es SOLO entrada. GPIO00 afecta al arranque del ESP32, usar con cuidado.

9. ADC (Conversor Analógico-Digital)

Lee voltajes analógicos. Importar con: `from hardware import ADC, Pin`

9.1 Constructor y métodos

Función / Atributo	Retorno	Descripción
<code>ADC(Pin(num), atten=)</code>	ADC	Crea ADC en un pin con atenuación
<code>.read()</code>	int	Lee valor raw (0-4095, 12 bits por defecto)
<code>.read_u16()</code>	int	Lee valor normalizado (0-65535, 16 bits)
<code>.read_uv()</code>	int	Lee voltaje en microvoltios
<code>.atten(atenuacion)</code>	void	Cambia atenuación
<code>.width(bits)</code>	void	Cambia resolución
<code>.init(atten=)</code>	void	Reinicializa con nueva config
<code>.block()</code>	object	Obtiene el bloque ADC subyacente

9.2 Constantes de ADC

Constante	Descripción
<code>ADC.ATTN_0DB</code>	Sin atenuación: 0 - 1.1V
<code>ADC.ATTN_2_5DB</code>	2.5dB: 0 - 1.5V
<code>ADC.ATTN_6DB</code>	6dB: 0 - 2.2V
<code>ADC.ATTN_11DB</code>	11dB: 0 - 3.3V (más común)
<code>ADC.WIDTH_9BIT</code>	Resolución 9 bits (0-511)
<code>ADC.WIDTH_10BIT</code>	Resolución 10 bits (0-1023)
<code>ADC.WIDTH_11BIT</code>	Resolución 11 bits (0-2047)
<code>ADC.WIDTH_12BIT</code>	Resolución 12 bits (0-4095) [defecto]

```
from hardware import ADC, Pin

# Sensor analógico en GPIO36 (rango completo 0-3.3V)
adc = ADC(Pin(36), atten=ADC.ATTN_11DB)

# Diferentes formas de leer
raw = adc.read()          # 0-4095
norm = adc.read_u16()     # 0-65535
uv = adc.read_uv()        # microvoltios

voltaje_v = uv / 1_000_000 # Convertir a voltios
print("Raw: {} Norm: {} Voltaje: {:.3f}V".format(raw, norm, voltaje_v))

# Cambiar atenuación dinámicamente
adc.atten(ADC.ATTN_6DB)   # Ahora 0-2.2V (más preciso en ese rango)
```


10. PWM (Modulación por Ancho de Pulso)

Genera señales PWM para LEDs, servos, motores. Importar con: `from hardware import PWM, Pin`

10.1 Métodos de PWM

Función / Atributo	Retorno	Descripción
<code>PWM(Pin(num), freq=, duty=)</code>	PWM	Crea PWM. freq en Hz, duty 0-1023
<code>.freq(valor)</code>	void/int	Establece/obtiene frecuencia (Hz)
<code>.duty(valor)</code>	void/int	Establece/obtiene ciclo de trabajo (0-1023)
<code>.duty_u16(valor)</code>	void/int	Ciclo de trabajo 16 bits (0-65535)
<code>.duty_ns(valor)</code>	void/int	Ciclo de trabajo en nanosegundos
<code>.init(freq=, duty=)</code>	void	Reinicializa con nueva config
<code>.deinit()</code>	void	Detiene PWM y libera el pin

```
from hardware import PWM, Pin

# LED con brillo variable en GPIO26
pwm_led = PWM(Pin(26), freq=1000, duty=0)

# Efecto fade in
for brillo in range(0, 1024, 10):
    pwm_led.duty(brillo)
    time.sleep_ms(20)

# Efecto fade out
for brillo in range(1023, -1, -10):
    pwm_led.duty(brillo)
    time.sleep_ms(20)

# Servo motor (50Hz, duty ~26-128 para 0-180°)
servo = PWM(Pin(26), freq=50, duty=77) # ~90°
servo.duty(26) # ~0°
servo.duty(128) # ~180°

# Leer configuración actual
print("Freq:", pwm_led.freq(), "Hz")
print("Duty:", pwm_led.duty())

# Liberar
pwm_led.deinit()
```

11. I2C (Bus de Comunicación)

Bus serie para sensores y periféricos. Importar con: `from hardware import I2C, Pin`

11.1 Métodos de I2C

Función / Atributo	Retorno	Descripción
<code>I2C(id, scl=, sda=, freq=)</code>	<code>I2C</code>	Crea bus I2C
<code>.scan()</code>	<code>list</code>	Lista direcciones de dispositivos conectados
<code>.readfrom(addr, n)</code>	<code>bytes</code>	Lee n bytes de la dirección addr
<code>.readfrom_into(addr, buf)</code>	<code>void</code>	Lee bytes directamente en buffer
<code>.readfrom_mem(addr, reg, n)</code>	<code>bytes</code>	Lee n bytes del registro reg
<code>.readfrom_mem_into(addr, reg, buf)</code>	<code>void</code>	Lee registro en buffer
<code>.writeto(addr, data)</code>	<code>void</code>	Escribe data a la dirección addr
<code>.writeto_mem(addr, reg, data)</code>	<code>void</code>	Escribe data en registro reg
<code>.writevto(addr, vectors)</code>	<code>void</code>	Escritura vectorizada (múltiples buffers)
<code>.start()</code>	<code>void</code>	Envía condición START (bajo nivel)
<code>.stop()</code>	<code>void</code>	Envía condición STOP (bajo nivel)
<code>.readinto(buf)</code>	<code>void</code>	Lee bytes al buffer (bajo nivel)
<code>.write(data)</code>	<code>int</code>	Escribe bytes (bajo nivel)
<code>.init(scl=, sda=, freq=)</code>	<code>void</code>	Reinicializa con nueva config

```
from hardware import I2C, Pin

# Port A del M5StickC Plus 2
i2c = I2C(0, scl=Pin(33), sda=Pin(32), freq=100000)

# Escanear dispositivos
devices = i2c.scan()
print("Dispositivos:", [hex(d) for d in devices])

# Leer registro de un sensor (ej: dirección 0x57, registro 0x00)
data = i2c.readfrom_mem(0x57, 0x00, 3)
print("Datos:", [hex(b) for b in data])

# Escribir comando a un sensor
i2c.writeto_mem(0x57, 0x01, bytes([0x03]))

# Lectura directa (sin registro)
raw = i2c.readfrom(0x57, 6)

# Ejemplo con sensor RCWL-9620 (ultrasónico I2C)
i2c.writeto(0x57, bytes([0x01])) # Trigger medida
time.sleep_ms(120)
data = i2c.readfrom(0x57, 3)
distancia_mm = (data[0] << 16 | data[1] << 8 | data[2]) / 1000
print("Distancia: {:.1f} mm".format(distancia_mm))
```

✓ **TIP:** Port A del M5StickC Plus 2: GPIO33 (SCL) y GPIO32 (SDA). Frecuencia estándar: 100kHz, rápida: 400kHz.

12. UART (Puerto Serie)

Comunicación serie asíncrona. Importar con: `from hardware import UART, Pin`

12.1 Métodos de UART

Función / Atributo	Retorno	Descripción
<code>UART(id, tx=, rx=, baudrate=)</code>	UART	Crea puerto serie
<code>.read(n)</code>	bytes/None	Lee hasta n bytes (None si no hay datos)
<code>.readinto(buf)</code>	int/None	Lee directamente en buffer
<code>.readline()</code>	bytes/None	Lee hasta <code>\n</code>
<code>.write(data)</code>	int	Escribe datos, devuelve bytes escritos
<code>.any()</code>	int	Bytes disponibles para leer
<code>.flush()</code>	void	Espera a que se envíen todos los datos
<code>.txdone()</code>	bool	True si la transmisión ha terminado
<code>.sendbreak()</code>	void	Envía condición BREAK
<code>.irq(handler, trigger, hard)</code>	void	Configura interrupción RX
<code>.init(...)</code>	void	Reinicializa con nueva config
<code>.deinit()</code>	void	Libera el puerto

12.2 Constantes de UART

Constante	Descripción
<code>UART.IRQ_RX</code>	Interrupción al recibir datos
<code>UART.IRQ_RXIDLE</code>	Interrupción cuando RX queda inactivo
<code>UART.IRQ_BREAK</code>	Interrupción en condición BREAK
<code>UART.INV_TX / INV_RX</code>	Invertir señal TX / RX
<code>UART.INV_CTS / INV_RTS</code>	Invertir señales de control de flujo
<code>UART.RTS / CTS</code>	Pines de control de flujo
<code>UART.MODE_UART</code>	Modo UART estándar
<code>UART.MODE_RS485_HALF_DUPLEX</code>	Modo RS-485 half duplex
<code>UART.MODE_IRDA</code>	Modo infrarrojo IrDA

```
from hardware import UART, Pin

# Puerto serie en GPIO26 (TX) y GPIO36 (RX)
uart = UART(1, tx=Pin(26), rx=Pin(36), baudrate=9600)

# Enviar datos
uart.write("Hola desde M5\n")
uart.write(bytes([0x01, 0x02, 0x03]))

# Esperar a que termine el envío
uart.flush()

# Recibir datos
if uart.any() > 0:
    datos = uart.read(uart.any())
    print("Recibido:", datos.decode())

# Leer línea completa
linea = uart.readline()
```

```
if linea:
    print("Línea:", linea.decode().strip())

# Interrupción al recibir
def on_rx(uart_obj):
    data = uart_obj.read(uart_obj.any())
    print("IRQ RX:", data)

uart.irq(on_rx, UART.IRQ_RXIDLE)

# Liberar
uart.deinit()
```

13. Temporización (time)

Módulo completo de temporización y medida de tiempo.

13.1 Funciones de pausa

Función / Atributo	Retorno	Descripción
<code>time.sleep(s)</code>	void	Pausa en segundos (acepta decimales)
<code>time.sleep_ms(ms)</code>	void	Pausa en milisegundos
<code>time.sleep_us(us)</code>	void	Pausa en microsegundos

13.2 Funciones de ticks (temporización precisa)

Función / Atributo	Retorno	Descripción
<code>time.ticks_ms()</code>	int	Contador en milisegundos (se desborda)
<code>time.ticks_us()</code>	int	Contador en microsegundos
<code>time.ticks_cpu()</code>	int	Contador de ciclos de CPU
<code>time.ticks_diff(a, b)</code>	int	Diferencia a - b (maneja desbordamiento)
<code>time.ticks_add(ticks, delta)</code>	int	Suma delta a ticks (maneja desbordamiento)

13.3 Funciones de fecha/hora

Función / Atributo	Retorno	Descripción
<code>time.time()</code>	int	Segundos desde epoch (1/1/2000 en MicroPython)
<code>time.time_ns()</code>	int	Nanosegundos desde epoch
<code>time.localtime()</code>	tuple	(año, mes, día, hora, min, seg, día_semana, día_año)
<code>time.gmtime(secs)</code>	tuple	Convierte timestamp a tupla UTC
<code>time.mktime(tuple)</code>	int	Convierte tupla a timestamp
<code>time.timezone()</code>	---	Zona horaria configurada

```
import time

# Pausas
time.sleep(2)           # 2 segundos
time.sleep(0.5)         # 0.5 segundos
time.sleep_ms(500)      # 500 ms
time.sleep_us(100)      # 100 µs

# Medir duración con precisión
t0 = time.ticks_us()
# ... operación a medir ...
t1 = time.ticks_us()
print("Duró: {} µs".format(time.ticks_diff(t1, t0)))

# Ejecutar algo cada 2 segundos SIN bloquear
ultimo = time.ticks_ms()
def loop():
    global ultimo
    M5.update()
    ahora = time.ticks_ms()
    if time.ticks_diff(ahora, ultimo) >= 2000:
        ultimo = ahora
        print("Tick cada 2s!")
```

```
# Calcular deadline futuro
deadline = time.ticks_add(time.ticks_ms(), 5000) # 5s en el futuro

# Fecha y hora (requiere NTP o RTC)
t = time.localtime()
print("{:04d}-{:02d}-{:02d} {:02d}:{:02d}:{:02d}".format(
    t[0], t[1], t[2], t[3], t[4], t[5]))
```

✓ **TIP:** SIEMPRE usar `time.ticks_diff()` en vez de restar directamente. Los ticks se desbordan y la resta simple da resultados erróneos.

14. WiFi (network)

Conexión inalámbrica a redes WiFi.

14.1 Métodos de WLAN

Función / Atributo	Retorno	Descripción
network.WLAN(network.STA_IF)	WLAN	Modo estación (conectarse a router)
network.WLAN(network.AP_IF)	WLAN	Modo punto de acceso
.active(True/False)	void/bool	Activa/desactiva, o consulta estado
.connect(ssid, password)	void	Inicia conexión a red
.disconnect()	void	Desconecta de la red
.isconnected()	bool	True si está conectado con IP
.ifconfig()	tuple	(IP, máscara, gateway, DNS)
.status()	int	Código de estado de la conexión
.scan()	list	Lista redes WiFi disponibles
.config(...)	varies	Configura parámetros (ssid, channel, etc)

14.2 Funciones globales de network

Función / Atributo	Retorno	Descripción
network.hostname(name)	void/str	Establece/obtiene hostname
network.country(code)	void/str	Establece/obtiene código de país
network.ipconfig(key)	varies	Configuración IP global
network.phy_mode(mode)	void/int	Modo PHY: 11B, 11G, 11N, LR

14.3 Constantes de estado

Constante	Descripción
network.STAT_IDLE	Inactivo
network.STAT_CONNECTING	Conectando
network.STAT_GOT_IP	Conectado con IP
network.STAT_NO_AP_FOUND	No se encontró la red
network.STAT_WRONG_PASSWORD	Contraseña incorrecta
network.STAT_BEACON_TIMEOUT	Timeout de beacon
network.STAT_ASSOC_FAIL	Fallo de asociación
network.STAT_CONNECT_FAIL	Fallo de conexión
network.STAT_HANDSHAKE_TIMEOUT	Timeout en handshake

14.4 Constantes de autenticación

Constante	Descripción
network.AUTH_OPEN	Red abierta (sin contraseña)
network.AUTH_WEP	WEP (inseguro, legacy)
network.AUTH_WPA_PSK	WPA-PSK

network.AUTH_WPA2_PSK	WPA2-PSK (más común)
network.AUTH_WPA_WPA2_PSK	WPA/WPA2 mixto
network.AUTH_WPA3_PSK	WPA3-PSK
network.AUTH_WPA2_WPA3_PSK	WPA2/WPA3 mixto
network.AUTH_WPA2_ENTERPRISE	WPA2 Enterprise (802.1X)

Conexión WiFi robusta (patrón recomendado)

```
import network
import time

def connect_wifi(ssid, password, timeout_s=20):
    wlan = network.WLAN(network.STA_IF)

    # Desactivar AP
    ap = network.WLAN(network.AP_IF)
    ap.active(False)

    # RESET OBLIGATORIO (evita OSError: Wifi Internal Error)
    wlan.active(False)
    time.sleep(1)
    wlan.active(True)
    time.sleep(1)

    if not wlan.isconnected():
        wlan.connect(ssid, password)
        t = 0
        while not wlan.isconnected():
            time.sleep_ms(500)
            t += 1
            if t > timeout_s * 2:
                return None

    ip = wlan.ifconfig()[0]
    return ip

# Uso
ip = connect_wifi("MI_RED", "MI_PASSWORD")
if ip:
    print("Conectado:", ip)
else:
    print("Fallo WiFi")

# Escanear redes disponibles
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
redes = wlan.scan()
for r in redes:
    print("SSID: {} RSSI: {} Auth: {}".format(r[0].decode(), r[3], r[4]))
```

⚠ SIEMPRE hacer active(False), sleep(1), active(True), sleep(1) antes de connect(). Sin este reset se obtiene 'OSError: Wifi Internal Error'.

15. Sockets (TCP/UDP)

Comunicación de red de bajo nivel.

15.1 Métodos de socket

Función / Atributo	Retorno	Descripción
socket.socket(family, type)	socket	Crea socket
.bind((ip, port))	void	Asocia a dirección/puerto
.listen(backlog)	void	Escucha conexiones (TCP)
.accept()	tuple	(socket_cliente, dirección)
.connect((ip, port))	void	Conecta a servidor (TCP)
.send(data)	int	Envía datos (TCP)
.recv(bufsize)	bytes	Recibe datos (TCP)
.sendto(data, (ip, port))	int	Envía datos (UDP)
.recvfrom(bufsize)	tuple	(datos, (ip, port))
.settimeout(s)	void	Timeout en segundos (None=bloquear)
.setblocking(flag)	void	True=bloqueante, False=no bloqueante
.setsockopt(level, opt, val)	void	Opciones del socket
.close()	void	Cierra el socket

15.2 Constantes

Constante	Descripción
socket.AF_INET	IPv4
socket.AF_INET6	IPv6
socket.SOCK_STREAM	TCP
socket.SOCK_DGRAM	UDP
socket.SOCK_RAW	Raw socket
socket.SOL_SOCKET	Nivel de opciones de socket
socket.SO_REUSEADDR	Reutilizar dirección
socket.SO_BROADCAST	Permitir broadcast
socket.SO_BINDTODEVICE	Vincular a interfaz específica
socket.IPPROTO_TCP / UDP / IP	Protocolos
socket.TCP_NODELAY	Deshabilitar algoritmo de Nagle
socket.IP_ADD_MEMBERSHIP	Unirse a grupo multicast
socket.getaddrinfo(host, port)	Resolución DNS

Ejemplo: Envío UDP rápido

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
target = ("192.168.1.100", 5005)

# Enviar texto
sock.sendto("Hola\n".encode(), target)

# Enviar datos binarios del IMU
```

```
import struct
data = struct.pack('<I3f', time.ticks_ms(), *Imu.getAccel())
sock.sendto(data, target)

sock.close()
```

Ejemplo: Servidor HTTP en el M5StickC

```
import socket

srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind(('', 80))
srv.listen(1)
srv.settimeout(0.5)

while True:
    M5.update()
    try:
        cl, addr = srv.accept()
        req = cl.recv(1024).decode()

        # Parsear método y ruta
        path = req.split(" ")[1] if " " in req else "/"

        acc = Imu.getAccel()
        html = "<html><body>"
        html += "<h2>M5StickC Plus 2</h2>"
        html += "<p>Acc: {:.2f}, {:.2f}, {:.2f}</p>".format(*acc)
        html += "<p>Bat: {}%</p>".format(Power.getBatteryLevel())
        html += "</body></html>"

        cl.send("HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n")
        cl.send(html)
        cl.close()
    except OSError:
        pass # Timeout normal en MicroPython
```

✓ **TIP:** En MicroPython los timeouts de socket lanzan OSError, NO socket.timeout.

16. MQTT

Protocolo publish/subscribe para IoT. Librería umqtt incluida en UIFlow 2.0.

16.1 Constructor y métodos de MQTTClient

Función / Atributo	Retorno	Descripción
MQTTClient(id, server, port=, user=, password=, keepalive=, ssl=)	MQTTClient	Crea cliente MQTT
.connect()	void	Conecta al broker
.disconnect()	void	Desconecta del broker
.isconnected()	bool	True si está conectado
.reconnect()	void	Reconecta al broker
.publish(topic, msg, qos=0, retain=False)	void	Publica mensaje en un topic
.subscribe(topic, qos=0)	void	Se suscribe a un topic
.unsubscribe(topic)	void	Cancela suscripción
.set_callback(func)	void	Define función para mensajes recibidos
.check_msg()	void	Comprueba mensajes (no bloqueante)
.wait_msg()	void	Espera un mensaje (BLOQUEANTE)
.ping()	void	Envía ping al broker (keep alive)
.set_last_will(topic, msg, retain, qos)	void	Define testamento (LWT)

16.2 Atributos de configuración

Función / Atributo	Retorno	Descripción
.DEBUG	bool	Activa mensajes de depuración
.DELAY	int	Delay entre operaciones
.log()	---	Función de log interna
.delay(ms)	void	Pausa interna

⚠ Usar SIEMPRE keyword arguments: port=, user=, password=, ssl=. Los argumentos posicionales causan errores.

Ejemplo: Publicar con SSL (HiveMQ Cloud)

```
from umqtt import MQTTClient
import ssl

# Variante 1: SSLContext (firmware >= 2.3.x)
context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
context.verify_mode = ssl.CERT_NONE

client = MQTTClient(
    "m5stick_01",
    "tu-cluster.s1.eu.hivemq.cloud",
    port=8883,
    user="usuario",
    password="password",
    keepalive=60,
    ssl=context
)
```

```
# Variante 2: ssl_params (firmware antiguo)
# client = MQTTClient(
#     "m5stick_01", "tu-cluster.hivemq.cloud",
#     port=8883, user="usuario", password="password",
#     keepalive=60, ssl=True,
#     ssl_params={"server_hostname": "tu-cluster.hivemq.cloud"})

# Definir LWT (Last Will and Testament)
client.set_last_will("m5stick/status", "offline", retain=True, qos=1)

client.connect()
client.publish("m5stick/status", "online", retain=True)
client.publish("m5stick/datos", '{"temp": 23.5}')
client.disconnect()
```

Ejemplo: Suscribirse y recibir

```
def on_message(topic, msg):
    topic_str = topic.decode() if isinstance(topic, bytes) else str(topic)
    msg_str = msg.decode() if isinstance(msg, bytes) else str(msg)
    print("Topic:", topic_str, "Msg:", msg_str)
    label_rx.setText(msg_str[:22])

client.set_callback(on_message)
client.connect()
client.subscribe("m5stick/cmd", 0)

# En el loop (no bloqueante)
def loop():
    M5.update()
    try:
        client.check_msg()
    except:
        try:
            client.reconnect()
        except:
            pass
    time.sleep_ms(100)
```

17. SSL / TLS

Módulo para conexiones seguras encriptadas.

17.1 Funciones y constantes

Función / Atributo	Retorno	Descripción
<code>ssl.SSLContext(protocol)</code>	<code>SSLContext</code>	Crea contexto SSL
<code>ssl.wrap_socket(sock, ...)</code>	<code>SSLSocket</code>	Envuelve socket con SSL (legacy)
<code>ssl.PROTOCOL_TLS_CLIENT</code>	<code>int</code>	Protocolo TLS para cliente
<code>ssl.PROTOCOL_TLS_SERVER</code>	<code>int</code>	Protocolo TLS para servidor
<code>ssl.PROTOCOL_DTLS_CLIENT</code>	<code>int</code>	DTLS para cliente (datagramas)
<code>ssl.PROTOCOL_DTLS_SERVER</code>	<code>int</code>	DTLS para servidor
<code>ssl.CERT_NONE</code>	<code>int</code>	No verificar certificado del servidor
<code>ssl.CERT_OPTIONAL</code>	<code>int</code>	Verificación opcional
<code>ssl.CERT_REQUIRED</code>	<code>int</code>	Verificación obligatoria
<code>ssl.MBEDTLS_VERSION</code>	<code>str</code>	Versión de la librería mbedTLS

```
import ssl

# Contexto para cliente TLS (MQTT, HTTPS)
ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
ctx.verify_mode = ssl.CERT_NONE # No verificar certificado

# Usar con MQTT
# client = MQTTClient(..., ssl=ctx)

# Usar con socket directo (HTTPS)
import socket
sock = socket.socket()
sock.connect(("httpbin.org", 443))
ssock = ctx.wrap_socket(sock, server_hostname="httpbin.org")
ssock.write(b"GET / HTTP/1.0\r\nHost: httpbin.org\r\n\r\n")
response = ssock.read(1024)
ssock.close()

print("mbedTLS:", ssl.MBEDTLS_VERSION)
```

18. Sistema de Archivos (os)

18.1 Métodos de os

Función / Atributo	Retorno	Descripción
os.listdir(path)	list	Lista archivos en directorio
os.ilistdir(path)	iterator	Lista como iterador (ahorra RAM)
os.stat(path)	tuple	Info del archivo. [6]=tamaño bytes
os.statvfs(path)	tuple	Info del sistema de archivos
os.getcwd()	str	Directorio actual
os.chdir(path)	void	Cambia directorio
os.mkdir(path)	void	Crea directorio
os.rmdir(path)	void	Borra directorio vacío
os.remove(path)	void	Borra archivo
os.unlink(path)	void	Borra archivo (alias de remove)
os.rename(old, new)	void	Renombra/mueve archivo
os.sync()	void	Fuerza escritura de buffers a flash
os.mount(device, path)	void	Monta sistema de archivos
os.umount(path)	void	Desmonta sistema de archivos
os.uname()	tuple	Info del sistema (versión, máquina...)
os.urandom(n)	bytes	Genera n bytes aleatorios
os.dupterm(stream)	void	Duplica terminal a un stream

18.2 Clases de sistema de archivos

Constante	Descripción
os.VfsFat	Sistema de archivos FAT
os.VfsLfs2	Sistema de archivos LittleFS v2

```
import os

# Listar archivos
print(os.listdir("/flash"))

# Info del sistema
print(os.uname()) # (sysname, nodename, release, version, machine)

# Espacio libre
stat = os.statvfs('/flash')
block_size = stat[0]
free_blocks = stat[3]
total_blocks = stat[2]
libre_kb = (block_size * free_blocks) // 1024
total_kb = (block_size * total_blocks) // 1024
print("Flash: {}/{} KB libres".format(libre_kb, total_kb))

# Tamaño de archivo
size = os.stat("/flash/boot.py")[6]
print("boot.py:", size, "bytes")

# Generar bytes aleatorios (para IDs únicos)
rand = os.urandom(4)
uid = ''.join(['{:02x}'.format(b) for b in rand])
```

```
print("UID:", uid)

# Lectura/escritura de archivos (built-in, no os)
with open("/flash/datos.csv", "w") as f:
    f.write("ts,valor\n")
    f.write("1000,23.5\n")

with open("/flash/datos.csv", "a") as f: # Append
    f.write("2000,24.1\n")

with open("/flash/datos.csv", "r") as f:
    for line in f:
        print(line.strip())

os.remove("/flash/datos.csv")
```

19. JSON

Función / Atributo	Retorno	Descripción
<code>json.dumps(obj)</code>	str	Objeto Python → texto JSON
<code>json.dump(obj, stream)</code>	void	Objeto Python → JSON a archivo/stream
<code>json.loads(text)</code>	dict/list	Texto JSON → objeto Python
<code>json.load(stream)</code>	dict/list	JSON desde archivo/stream → objeto Python

```
import json

# Serializar
datos = {"dispositivo": "M5StickCPlus2", "temp": 23.5, "activo": True}
texto = json.dumps(datos)
print(texto) # '{"dispositivo":"M5StickCPlus2","temp":23.5,"activo":true}'

# Deserializar
recibido = '{"cmd": "led", "estado": true, "brillo": 128}'
obj = json.loads(recibido)
print(obj["cmd"]) # "led"
print(obj["brillo"]) # 128

# Guardar/cargar desde archivo
with open("/flash/config.json", "w") as f:
    json.dump({"wifi_ssid": "MiRed", "intervalo": 5}, f)

with open("/flash/config.json", "r") as f:
    config = json.load(f)
    print(config["wifi_ssid"])
```

✓ **TIP:** Para datos simples, construir JSON con `format()` es más eficiente en RAM que `json.dumps()`.

20. Struct (Datos Binarios)

Empaquetar/desempaquetar datos binarios. Esencial para comunicación eficiente por UDP.

20.1 Funciones

Función / Atributo	Retorno	Descripción
<code>struct.pack(fmt, v1, v2...)</code>	bytes	Empaqueta valores según formato
<code>struct.unpack(fmt, data)</code>	tuple	Desempaqueta bytes a valores
<code>struct.pack_into(fmt, buf, offset, v...)</code>	void	Empaqueta en buffer existente
<code>struct.unpack_from(fmt, buf, offset)</code>	tuple	Desempaqueta desde offset
<code>struct.calcsize(fmt)</code>	int	Tamaño del formato en bytes

20.2 Códigos de formato

Código	Tamaño	Tipo Python
'<'	Prefijo	Little-endian (usar siempre)
'>'	Prefijo	Big-endian
'b' / 'B'	1 byte	int8 con/sin signo
'h' / 'H'	2 bytes	int16 con/sin signo
'i' / 'I'	4 bytes	int32 con/sin signo
'l' / 'L'	4 bytes	long con/sin signo
'q' / 'Q'	8 bytes	int64 con/sin signo
'f'	4 bytes	float32
'd'	8 bytes	float64 (double)
'?'	1 byte	bool
'Ns'	N bytes	string de N bytes

```
import struct

# Empaquetar IMU: timestamp + 6 floats
ts = time.ticks_ms()
acc = Imu.getAccel()
gyro = Imu.getGyro()
data = struct.pack('<I6f', ts,
    acc[0], acc[1], acc[2],
    gyro[0], gyro[1], gyro[2])
print("Bytes:", len(data)) # 28 bytes

# Desempaquetar
ts2, ax, ay, az, gx, gy, gz = struct.unpack('<I6f', data)

# Calcular tamaño
print(struct.calcsize('<I6f')) # 28
print(struct.calcsize('<3f')) # 12

# Pack/unpack con buffer preasignado (más eficiente)
buf = bytearray(28)
struct.pack_into('<I6f', buf, 0, ts, *acc, *gyro)
vals = struct.unpack_from('<I6f', buf, 0)
```

21. Math (Matemáticas)

Referencia completa de todas las funciones matemáticas disponibles.

21.1 Constantes

Función / Atributo	Retorno	Descripción
<code>math.pi</code>	float	3.141592653589793
<code>math.e</code>	float	2.718281828459045
<code>math.tau</code>	float	6.283185307179586 (2π)
<code>math.inf</code>	float	Infinito positivo
<code>math.nan</code>	float	Not a Number

21.2 Funciones básicas

Función / Atributo	Retorno	Descripción
<code>math.fabs(x)</code>	float	Valor absoluto
<code>math.ceil(x)</code>	int	Redondeo hacia arriba
<code>math.floor(x)</code>	int	Redondeo hacia abajo
<code>math.trunc(x)</code>	int	Trunca parte decimal
<code>math.fmod(x, y)</code>	float	Módulo (resto de x/y)
<code>math.modf(x)</code>	tuple	(parte_decimal, parte_entera)
<code>math.copysign(x, y)</code>	float	x con el signo de y
<code>math.factorial(n)</code>	int	Factorial de n
<code>math.pow(x, y)</code>	float	x elevado a y
<code>math.sqrt(x)</code>	float	Raíz cuadrada
<code>math.isclose(a, b, rel_tol, abs_tol)</code>	bool	Compara si $a \approx b$
<code>math.isfinite(x)</code>	bool	True si x es finito
<code>math.isinf(x)</code>	bool	True si x es infinito
<code>math.isnan(x)</code>	bool	True si x es NaN

21.3 Funciones trigonométricas (radianes)

Función / Atributo	Retorno	Descripción
<code>math.sin(x)</code>	float	Seno
<code>math.cos(x)</code>	float	Coseno
<code>math.tan(x)</code>	float	Tangente
<code>math.asin(x)</code>	float	Arcoseno ($-1 \leq x \leq 1$)
<code>math.acos(x)</code>	float	Arcocoseno ($-1 \leq x \leq 1$)
<code>math.atan(x)</code>	float	Arcotangente
<code>math.atan2(y, x)</code>	float	Arcotangente de y/x (4 cuadrantes)
<code>math.degrees(rad)</code>	float	Radianes $\rightarrow$ grados
<code>math.radians(deg)</code>	float	Grados $\rightarrow$ radianes

21.4 Funciones hiperbólicas

Función / Atributo	Retorno	Descripción
<code>math.sinh(x)</code>	float	Seno hiperbólico
<code>math.cosh(x)</code>	float	Coseno hiperbólico
<code>math.tanh(x)</code>	float	Tangente hiperbólica
<code>math.asinh(x)</code>	float	Arcoseno hiperbólico
<code>math.acosh(x)</code>	float	Arcocoseno hiperbólico
<code>math.atanh(x)</code>	float	Arcotangente hiperbólica

21.5 Funciones logarítmicas y exponenciales

Función / Atributo	Retorno	Descripción
<code>math.log(x)</code>	float	Logaritmo natural (base e)
<code>math.log(x, base)</code>	float	Logaritmo en base arbitraria
<code>math.log2(x)</code>	float	Logaritmo base 2
<code>math.log10(x)</code>	float	Logaritmo base 10
<code>math.exp(x)</code>	float	e elevado a x
<code>math.expm1(x)</code>	float	$\exp(x) - 1$ (preciso para x pequeño)
<code>math.ldexp(x, i)</code>	float	$x * 2^i$
<code>math.frexp(x)</code>	tuple	(mantisa, exponente) tal que $x = m * 2^e$

21.6 Funciones especiales

Función / Atributo	Retorno	Descripción
<code>math.gamma(x)</code>	float	Función Gamma
<code>math.lgamma(x)</code>	float	Log natural de Gamma(x)
<code>math.erf(x)</code>	float	Función error
<code>math.erfc(x)</code>	float	Función error complementaria ($1 - \text{erf}(x)$)

```
import math

# Conversión de ángulos
ang_deg = 45
ang_rad = math.radians(ang_deg) # 0.7854
print("sin(45°) =", math.sin(ang_rad)) # 0.7071

# Inclínómetro con IMU
acc = Imu.getAccel()
pitch = math.degrees(math.atan2(acc[0], math.sqrt(acc[1]**2 + acc[2]**2)))
roll = math.degrees(math.atan2(acc[1], math.sqrt(acc[0]**2 + acc[2]**2)))

# Magnitud de un vector
mag = math.sqrt(acc[0]**2 + acc[1]**2 + acc[2]**2)

# Comparar floats con tolerancia
if math.isclose(mag, 1.0, abs_tol=0.05):
    print("En reposo (1g)")

# Decibelios
valor_raw = 2048
db = 20 * math.log10(valor_raw / 4095)
```

```
# Redondeos
print(math.ceil(3.2))    # 4
print(math.floor(3.8))   # 3
print(math.trunc(-3.7))  # -3
```

22. Gestión de Memoria (gc)

El ESP32 tiene ~520 KB de RAM. Es importante gestionar la memoria en programas largos.

Función / Atributo	Retorno	Descripción
<code>gc.collect()</code>	void	Fuerza recolección de basura (libera RAM)
<code>gc.mem_free()</code>	int	Bytes de RAM libres
<code>gc.mem_alloc()</code>	int	Bytes de RAM usados
<code>gc.enable()</code>	void	Habilita recolección automática
<code>gc.disable()</code>	void	Deshabilita recolección automática
<code>gc.isenabled()</code>	bool	True si la recolección automática está activa
<code>gc.threshold(amount)</code>	void/int	Establece/obtiene umbral de recolección automática

```
import gc

# Ver estado de memoria
print("Libre:", gc.mem_free(), "bytes")
print("Usado:", gc.mem_alloc(), "bytes")
print("Auto GC:", gc.isenabled())

# Forzar liberación
gc.collect()
print("Libre tras gc:", gc.mem_free(), "bytes")

# Configurar umbral: recolectar cuando quede poco
gc.threshold(gc.mem_free() // 4) # GC cuando se use 75%

# Patrón recomendado en bucle largo
count = 0
def loop():
    global count
    M5.update()
    count += 1

    # GC cada 100 iteraciones
    if count % 100 == 0:
        gc.collect()

# GC antes de operaciones que consumen RAM
if BtnA.wasPressed():
    gc.collect() # Liberar antes de conectar WiFi
    connect_wifi("ssid", "pass")
```

✓ **TIP:** Llama a `gc.collect()` antes de WiFi, MQTT, abrir archivos grandes, y periódicamente en bucles largos.

23. Referencia de Pines del M5StickC Plus 2

GPIO	Ubicación	Función	Notas
GPIO32	Port A (Grove)	SDA	I2C datos
GPIO33	Port A (Grove)	SCL	I2C reloj
GPIO26	Hat	GPIO/PWM/DAC	Salida general
GPIO36	Hat	ADC (solo IN)	Solo entrada, no soporta salida
GPIO0	Hat	GPIO	Afecta boot, usar con cuidado
GPIO25	Interno	Speaker	NO usar externamente
GPIO19	Interno	USB	NO usar externamente
GPIO4	Interno	IR LED	Emisor infrarrojo
GPIO35	Interno	Micrófono	NO usar externamente

24. Errores Comunes y Soluciones

24.1 OSError: Wifi Internal Error

Causa: Módulo WiFi en estado inconsistente al iniciar.

Solución: Siempre hacer `wlan.active(False)`, `time.sleep(1)`, `wlan.active(True)`, `time.sleep(1)` antes de `connect()`.

24.2 function takes N positional arguments but M were given

Causa: Pasar argumentos sin nombre a MQTTClient.

Solución: Usar keyword arguments: `port=8883`, `user='...'`, `password='...'`, `ssl=ctx`

24.3 OSError en socket (timeout)

Causa: MicroPython no tiene `socket.timeout` como Python estándar.

Solución: Capturar `OSError` en vez de `socket.timeout`.

24.4 MemoryError

Causa: RAM agotada (ESP32 tiene ~520 KB).

Solución: Llamar `gc.collect()`, reducir buffers, escribir a archivo en vez de acumular en listas, cerrar conexiones que no se usen.

24.5 La pantalla no se actualiza

Causa: Falta `M5.update()` en el loop, o el loop está bloqueado por un `sleep` largo.

Solución: Usar `time.sleep_ms()` con valores pequeños y llamar `M5.update()` siempre al inicio del loop.

24.6 ImportError: no module named 'Pin'

Causa: `from hardware import *` no exporta todas las clases en todas las versiones.

Solución: Importar individualmente: `from hardware import Pin, I2C, ADC, PWM, UART`

24.7 MQTT desconecta frecuentemente

Causa: No se envía `keepalive` o el intervalo es muy largo.

Solución: Usar `keepalive=60`, llamar `client.ping()` periódicamente, o implementar reconexión automática con `try/except`.